

# LEDGER — PROJECT OBJECTIVES SPECIFICATION

Enterprise-Grade Multi-Tenant Financial Tracking & Ingestion Engine

Architecture: PERN Stack (PostgreSQL, Express, React, Node)

Standard: IEEE 830 SRS Alignment

Document Status: Locked Specification v1.0

Security Standard: RLS + JWT + TOTP MFA

Execution Engine: Raw SQL + BullMQ + Redis

Target Audience: Core Development & QA Teams

## 1. EXECUTIVE SUMMARY & PROJECT INTENT

**Ledger** is an enterprise-ready, full-stack expense tracking and financial ledger system built on the PERN stack (PostgreSQL, Express.js, React, Node.js) and TypeScript. Guided by formal IEEE 830 Software Requirements Specification (SRS) standards, the project bridges the gap between individual personal finance tools and complex business accounting software.

The core vision of Ledger is to eliminate rigid single-account boundaries by enabling multi-tenant data isolation, asynchronous multi-format statement ingestion, real-time foreign exchange (FX) normalization, and automated receipt processing—all delivered through a cohesive visual identity inspired by traditional paper ledger books.

### CORE MISSION STATEMENT

To engineer a high-throughput, secure, and intuitive digital ledger platform that unifies manual entry, recurring automated billing, universal bank statement imports, and cloud-based receipt scanning into a single immutable transaction record.

## 2. PRIMARY PROJECT OBJECTIVES

The primary engineering and product objectives for the Ledger platform are structured across four core operational pillars:

### CORE 1. Unified Multi-Source Ingestion

Engine a singular, highly scalable transactional data pipeline where manual entries, recurring scheduled rules, universal bank CSV imports, and OCR-extracted receipt images feed into a unified transactional model without data fragmentation.

### SEC 2. Database Multi-Tenancy & Security

Enforce absolute data isolation at the PostgreSQL database level using native Row-Level Security (RLS), stateless JWT session tokens, AES-256 field encryption, TOTP Multi-Factor Authentication, and rate-limited authentication routes.

### DATA 3. Real-Time FX & Financial Analytics

Provide multi-currency conversion capabilities powered by scheduled FX rate fetching and Redis caching, allowing users to log transactions in foreign currencies while viewing consolidated aggregate reporting in their preferred base currency.

### SCALE 4. Asynchronous Queue Processing

Decouple CPU-intensive operations (Tesseract.js OCR receipt scanning, recurring transaction generation, FX synchronization) from Express HTTP threads using Redis-backed BullMQ message queues.

## 3. TARGET USER PERSONAS & OBJECTIVES

Ledger is architected to support multiple user profiles seamlessly within a single account, avoiding rigid persona silos:

| User Persona        | Primary Objectives & Needs                                                                                                                      | System Architectural Features                                                                                           |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Individual Consumer | Track personal day-to-day expenditures (rent, groceries, entertainment), set category spending limits, receive budget alert threshold warnings. | Personal category seeding, real-time budget progress calculation, instant transaction CRUD, intuitive visual dashboard. |

| User Persona                      | Primary Objectives & Needs                                                                                                                   | System Architectural Features                                                                                                              |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Small Business Owner / Freelancer | Segregate personal vs. business cash flows, track tax-deductible expenditures, generate audit-ready accounting PDF reports, upload receipts. | Multi-workspace switching, nested category trees with <code>tax_deductible</code> tags, PDFKit report exports, Cloudinary receipt uploads. |
| Students & Young Professionals    | Low-friction habit building, simple CSV bank statement imports, quick expense categorization without heavy enterprise setup.                 | Universal drag-and-drop CSV column mapper, automated MD5 deduplication queue, responsive UI.                                               |

## 4. FUNCTIONAL MODULES & TECHNICAL SCOPE

### Module 1: Authentication, Onboarding & Multi-Tenancy

- **Objective:** Deliver enterprise identity management and zero-trust data access.
- **Key Deliverables:** JWT access token sessions, bcrypt password salting, TOTP Multi-Factor Authentication ( `otplib` ), rate-limiting middleware, and PostgreSQL Row-Level Security (RLS) enforcement per request session.

### Module 2: Dynamic Workspaces & Hierarchical Categories

- **Objective:** Enable custom financial structures without hardcoded limits.
- **Key Deliverables:** Multi-workspace switching ( `Personal` , `Freelance` , `Property` ), nested parent-child category hierarchies, and configurable tax tags.

### Module 3: Core Transactions & FX Multi-Currency Engine

- **Objective:** Serve as the central relational anchor for all spending records.
- **Key Deliverables:** Raw SQL CRUD models, source tagging ( `manual` , `recurring` , `bank_statement` ), multi-currency schema, and background automated daily exchange rate synchronization with Redis caching.

### Module 4: Universal CSV Statement Ingestion & Deduplication

- **Objective:** Process heterogeneous bank statements from any global institution.
- **Key Deliverables:** Drag-and-drop interactive column mapper UI, streaming CSV parser ( `csv-parser` ), cryptographic hash generation ( `MD5(date + amount + payee)` ) for duplicate suppression, and manual review queues.

### Module 5: Cloud Receipt Management & Asynchronous OCR

- **Objective:** Automate document capture and receipt data extraction.
- **Key Deliverables:** Direct-to-cloud signed upload signatures using Cloudinary, asynchronous Redis/BullMQ background queue processing, and Tesseract.js OCR text parsing.

### Module 6: Audit Trail, Reporting & Compliance

- **Objective:** Ensure complete data transparency and regulatory compliance.
- **Key Deliverables:** PostgreSQL database triggers writing to an immutable `audit_logs` table, accounting PDF exports via `pdfkit` , CSV exports via `json2csv` , and GDPR self-service data wiping routes.

## 5. TECHNOLOGY STACK & DEPENDENCY ARCHITECTURE

| Layer                | Technology Choice         | Key Dependency Packages                                                  | Architectural Justification                                                                                            |
|----------------------|---------------------------|--------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Database             | PostgreSQL (Neon Cloud)   | <code>pg</code> (^8.22.0)                                                | Relational integrity, native Row-Level Security (RLS) policies, CTE analytical queries, and SQL database triggers.     |
| Backend API          | Node.js + Express.js + TS | <code>express</code> (^5.2.1), <code>zod</code> (^4.4.3)                 | RESTful routes, strict runtime Zod request validation, modular controller-service-model pattern.                       |
| Background Queue     | Redis + BullMQ            | <code>bullmq</code> (^6.0.6), <code>ioredis</code> (^6.0.0)              | Offloads asynchronous CPU-heavy tasks (OCR scanning, daily FX sync, recurring entries) without blocking HTTP handlers. |
| Media Storage        | Cloudinary SDK            | <code>cloudinary</code> (^2.10.0)                                        | Signed browser-to-cloud file uploads for physical receipts and statement documents without server memory buffers.      |
| Parsing & Processing | Tesseract.js & PDFKit     | <code>tesseract.js</code> , <code>pdfkit</code> , <code>pdf-parse</code> | Optical Character Recognition (OCR), statement PDF text extraction, and programmatic financial PDF generation.         |
| Security Layer       | JWT + Bcrypt + Helmet     | <code>jsonwebtoken</code> , <code>bcrypt</code> , <code>helmet</code>    | Cryptographic password hashing, stateless token management, security headers, and brute-force rate limiting.           |

## 6. QUALITY ASSURANCE & PERFORMANCE METRICS

To ensure financial calculation accuracy and reliable performance, Ledger defines strict QA objectives:

- **Unit Testing (Jest / ts-jest):** Achieve >85% test coverage across core mathematical calculations, budget threshold checks, FX rate conversions, and Zod input validation schemas.
- **API Integration Testing (Supertest):** Execute automated integration tests across all Express endpoints to verify route protection, status codes, and multi-tenant RLS workspace isolation.
- **Sub-100ms API Response Time:** Maintain sub-100ms response times for core transaction endpoints by caching aggregated dashboard analytics inside Redis.
- **Zero Data Bleed Guarantee:** Validate that database Row-Level Security policies strictly block cross-tenant data access across all API operations.

## 7. PROJECT BUILD SEQUENCE & MILESTONES

| Phase   | Milestone Focus         | Deliverables & Verification Criteria                                                                       |
|---------|-------------------------|------------------------------------------------------------------------------------------------------------|
| Phase 1 | Database & RLS Setup    | Execute Raw SQL migrations on Neon PostgreSQL; create tables, RLS policies, and audit triggers.            |
| Phase 2 | Auth & Session Security | Implement JWT auth routes, bcrypt hashing, TOTP MFA, and <code>setRlsContext</code> DB session middleware. |
| Phase 3 | Core Models & FX Engine | Build raw SQL transaction models, external FX exchange rate fetcher, and Redis rate lookup cache.          |

| Phase   | Milestone Focus     | Deliverables & Verification Criteria                                                                            |
|---------|---------------------|-----------------------------------------------------------------------------------------------------------------|
| Phase 4 | Ingestion & Workers | Deliver universal CSV parser with MD5 deduplication and BullMQ workers for Tesseract OCR receipt extraction.    |
| Phase 5 | React Frontend UI   | Build "Cream & Ink" visual components, workspace switcher, interactive CSV column mapper, and analytics graphs. |
| Phase 6 | Audit, Reports & QA | Generate PDF/CSV accounting reports, implement GDPR account wipe routes, and execute full Supertest suite.      |

**DOCUMENT SIGN-OFF & COMPLIANCE**

This document serves as the canonical project objectives specification for the **Ledger** development lifecycle. All feature implementations, database migrations, and API endpoint designs must strictly align with the security, performance, and structural standards outlined herein.